Лабораторна робота №9
Обробка графіки. Бібліотека GD

Крайняк Михайло Юрійович, 451 група

1. Мета і середовище

Мета: навчитися створювати растрові зображення засобами GD у PHP, будувати діаграми з даних, додавати текстовий водяний знак і повертати зображення у HTTP-відповіді; порівняти растрову PNG-діаграму з векторною SVG-версією.

Варіант 10 — розклад маршрутів автовокзалу. Власні дані — кількість відправлень з автовокзалу за напрямками на тиждень (вигадані числа).

PHP вебсервера	**8.2.12** (XAMPP). OSPanel і PHP 8.1 не встановлені. Код використовує лише можливості PHP 8.1 (`GdImage`, `array_is_list()`, тип `never`) і на 8.1 працює так само.
Файл конфігурації	`C:\xampp\php\php.ini` — це `Loaded Configuration File` з тимчасової сторінки `phpinfo()`, яку після перевірки видалено.
GD	У `php.ini` рядок 931 — `;extension=gd`, тобто розширення вимкнене, хоча `ext\php_gd.dll` є. Щоб не змінювати спільний `php.ini` XAMPP, GD вмикалося параметром `-d extension=gd` для вебсервера й для CLI. Це рівнозначно розкоментуванню рядка. Без параметра CLI повідомляє «Для цього прикладу потрібне розширення GD» (розділ 6, пункт 11): CLI і вебсервер можуть мати різну конфігурацію. `gd_info()`: **GD Version: bundled (2.1.0 compatible)**; **PNG Support: так**; **FreeType Support: так** (with freetype); також JPEG, GIF, WebP, AVIF.
Шрифт	**DejaVu Sans** (`C:/Windows/Fonts/DejaVuSans.ttf`, вільна ліцензія Bitstream Vera/DejaVu) — повний набір кирилиці. Шлях задається у `src/config.php` або змінною середовища `LAB09_FONT`, але не параметром запиту. Файл шрифту до подання не включено.
Адреса	`http://127.0.0.1:8090/lab-09/` — вбудований сервер PHP. Корінь сервера містить каталог `lab-09`, як `C:\xampp\htdocs\lab-09`.
Перевірка	`tools/checks.sh` (curl, Python/Pillow для аналізу пікселів, XML-парсер), Microsoft Edge (відображення SVG і сторінки), W3C Nu HTML Checker.

2. Файли: готові приклади і власні зміни

Файл	Походження	Призначення
`src/common.php`, `diagram_png_1.php`, `diagram_png_2.php`, `diagram_svg.php`, `diagram_1.svg`, `chart-data.json`, `watermark.php`, `prepare_sample.php`, `images/sample.png`	Готові, **без змін**	Приклади з методички: кругова й стовпчаста діаграми, SVG, водяний знак
`index.php` (корінь `lab-09`)	Власний	HTML-сторінка: готові PNG і SVG з таблицею даних, власні діаграми з таблицею, водяні знаки, кругова діаграма

Файл	Походження	Призначення
src/bus-data.json	Власні дані	7 напрямків: Київ 42, Черкаси 35, Сміла 28, Умань 21, Золотоноша 14, Кам'янець-Подільський 4, Біла Церква 9
src/config.php	Власний	Шрифт, шляхи до даних і зображень, підпис «група · вигадане ім’я · дата»
src/bus-common.php	Власний	Читання й перевірка JSON, спільна розкладка PNG/SVG, підбір кегля й перенесення підписів, текст TrueType
src/bus_png.php, src/bus_svg.php	Власні	Самостійні завдання 1 і 2: стовпчаста діаграма PNG і SVG з одного файла
src/watermark_bus.php, src/make_bus_image.php, src/images/bus.png	Власні	Самостійне завдання 3: водяний знак з українським підписом; генератор власного PNG 800×450
src/pie_png.php	Власний	Кругова діаграма зі змінними значеннями (крок 2.1)
src/watermark-output.png, src/watermark-bus-output.png	Результати	Створені командою CLI
tools/checks.sh → tools/checks-output.txt; tools/make_fixtures.php, tests/fixtures/data	Власні	Перевірки й тестові файли; негативні випадки застосовуються лише до копії src
results/w3c-validation.txt, results/*.png, results/*.svg	Результати	Валідація W3C, збережені PNG/SVG-відповіді

3. Масштаб і обмеження параметрів

Розміри полотна. GET-параметри `width` (400–1200, типово 640) і `height` (300–800, типово 400) перевіряє готова `queryInt()`:

- рядковий тип і `ctype_digit()` відкидають масиви, дробові, від’ємні значення, порожній рядок і пробіли;
- `FILTER_VALIDATE_INT` з `min_range`/`max_range` перевіряє межі; початкові нулі (`0640`) він теж відхиляє.

Помилка дає `InvalidArgumentException`, тобто HTTP 400 з текстовим поясненням. Межі обмежують пам’ять (найбільше полотно — 1,2×0,8 Мп) і лишають місце для підписів.

Висота стовпця. Область побудови займає від `top` = 70 до `bottom` = `height` − 78 (поля під заголовок, підписи категорій і підпис автора). Висота стовпця обчислюється так:

```
scale = max(1, max(value));   barHeight = round(value / scale × (bottom − top));   y = bottom − barHeight
```

Знаменник `max(1, …)` захищає від ділення на нуль, коли всі значення нульові: тоді всі висоти 0, а осі, нульова позначка й підписи лишаються. Ширину області `right - left` поділено на кількість категорій (`slot`). Проміжки по 15 % з кожного боку стовпця відкладаються *всередині* слота, тому останній стовпець не виходить за межі. Сітка проведена на чвертях шкали.

Розкладку `busLayout()` використовують і PNG, і SVG. Тому масштаб і порядок категорій однакові: висоти стовпців у SVG і висоти в пікселях PNG збігаються (розділ 6, пункт 5).

```
/**
 * Розкладка стовпців — та сама для PNG і SVG. Логіка barLayout() з готового прикладу,
 * але з полями для українських підписів, сітки та підпису автора.
 */
function busLayout(array $items, int $width, int $height): array
{
    $left = 64;
    $right = $width - 20;
    $top = 70;
    $bottom = $height - 78;
    $plotHeight = $bottom - $top;
    $slot = ($right - $left) / count($items);
    $max = max(array_column($items, 'value'));
    $scale = max(1, $max);                  // усі нулі — без ділення на нуль
    $bars = [];
    foreach ($items as $i => $row) {
        $x1 = (int) round($left + $i * $slot + $slot * 0.15);
        $x2 = (int) round($left + ($i + 1) * $slot - $slot * 0.15);
        $y = $bottom - (int) round($row['value'] / $scale * $plotHeight);
        $bars[] = ['x1' => $x1, 'x2' => $x2, 'cx' => (int) round(($x1 + $x2) / 2), 'y' => $y] + $row;
    }
    // Сітка: чверті шкали; підписи — округлені значення.
    $grid = [];
    foreach ([0, 1, 2, 3, 4] as $k) {
        $grid[] = ['y' => $bottom - (int) round($k / 4 * $plotHeight), 'value' => (int) round($k / 4 *
$scale)];
    }
    return ['left' => $left, 'right' => $right, 'top' => $top, 'bottom' => $bottom, 'slot' => $slot,
        'max' => $max, 'scale' => $scale, 'bars' => $bars, 'grid' => $max > 0 ? $grid : [$grid[0]]];
}
```

Підписи. Кегль GD задається в пунктах за 96 dpi, тому в SVG використано той самий розмір × 96/72. Ширину тексту в PNG вимірює `imagettfbbox()`, у SVG — наближена оцінка (0,58 em на символ). Для підпису категорії послідовно пробуються:

1. один рядок, кегль 11…9;
2. два рядки з розривом після пробілу чи дефіса;
3. скорочення з «…» кеглем 8.

Для всіх категорій береться спільний найменший кегль, а повні назви є в HTML-таблиці. Заголовок і підпис автора зменшуються до 9/8 пт, якщо не вміщуються.

```
/**
 * Підпис категорії під стовпцем: один рядок (кегль 11…9), два рядки з розривом після пробілу
 * чи дефіса (кегль 11…8) або скорочення з «…» (кегль 8). Повертає ['lines' => [...], 'size' => n].
 */
function fitLabel(string $text, float $maxWidth, callable $measure, int $maxSize = 11): array
```

```
{
    for ($size = $maxSize; $size >= min($maxSize, 9); $size--) {
        if ($measure($text, $size) <= $maxWidth) {
            return ['lines' => [$text], 'size' => $size];
        }
    }
    preg_match_all('/[ -]/u', $text, $breaks, PREG_OFFSET_CAPTURE);
    for ($size = min($maxSize, 11); $size >= 8; $size--) {
        foreach ($breaks[0] as [$char, $offset]) {
            $first = rtrim(substr($text, 0, $offset + ($char === '-' ? 1 : 0)));
            $second = ltrim(substr($text, $offset + 1));
            if ($first !== '' && $second !== '' && $measure($first, $size) <= $maxWidth &&
$measure($second, $size) <= $maxWidth) {
                return ['lines' => [$first, $second], 'size' => $size];
            }
        }
    }
    [$short, $size] = fitText($text, $maxWidth, $measure, 8, 8);
    return ['lines' => [$short], 'size' => $size];
}
```

4. Хід роботи

Кроки 1–2. Середовище і готові діаграми

Сторінка `diagram_png_1.php` будує кругову діаграму з частками 40/30/30. Кут кінця сектора обчислюється з накопиченої суми: `round(running / sum × 360)`, тож межі 144° і 252°, а останній сектор закінчується рівно на 360°. Скрипт перевірки обійшов коло з кроком 1° і порахував пікселі кожного кольору: 39–40 %, 30 %, 30 % (похибка — межі секторів між точками вибірки).

Щоб змінювати значення, створено власний `pie_png.php?values=…`:

- приймає 2–6 цілих 0–1000 через кому;
- нульова сума, масив, дробові чи від’ємні значення дають 400;
- сектор нульового розміру не малюється;
- легенда з відсотками виводиться шрифтом TrueType.

Наприклад, при `50,25,15,10` синій сектор займає половину кола (180°), а наступні межі — 270°, 324°, 360° (рис. 14).

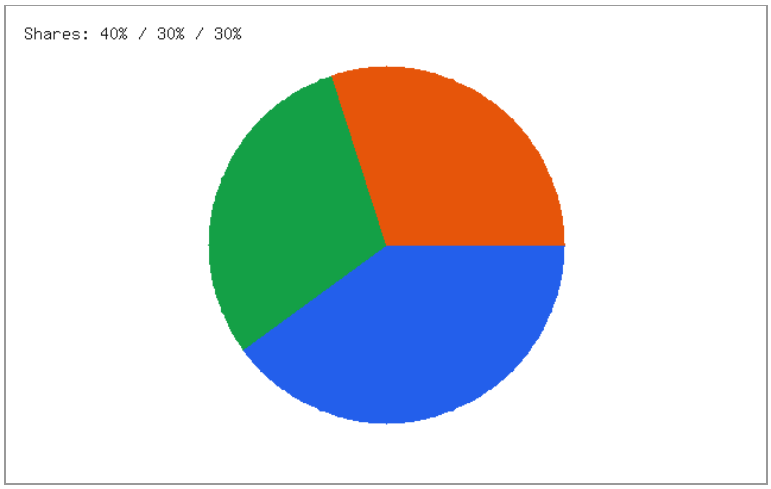


Рис. 1. Готова `diagram_png_1.php`: частки 40 %, 30 %, 30 %, PNG 640×400 без пошкоджень.

Готові `diagram_png_2.php` і `diagram_svg.php` читають `chart-data.json` (A–E: 20, 50, 30, 80, 40) через спільну `barLayout()`. Найвищий стовпець D — 300 px. Стовпець C має 113 px, тобто 113/300 = 0,377 ≈ 3/8 з похибкою округлення.

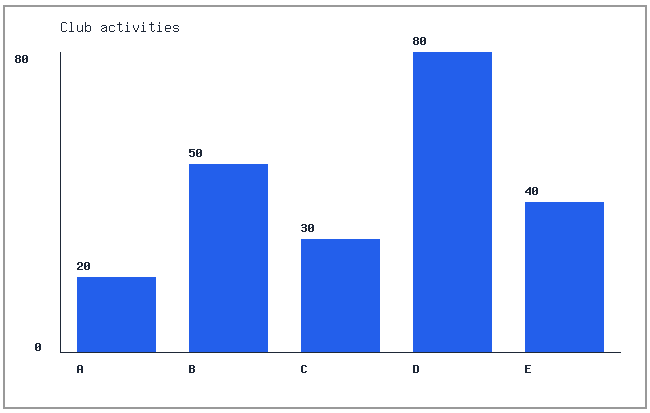


Рис. 2. `diagram_png_2.php` — PNG.

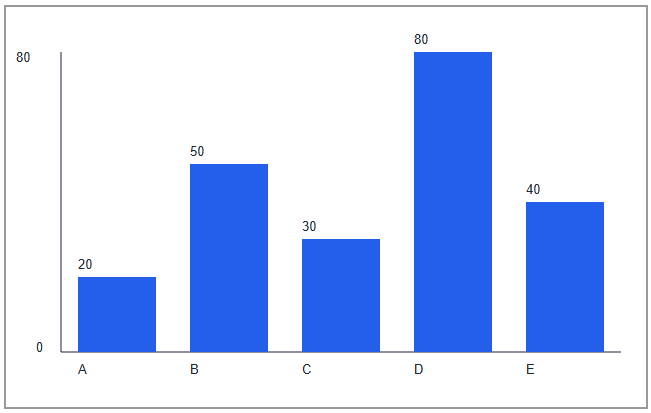


Рис. 3. `diagram_svg.php` — SVG у браузері.

Порівняння відповідей (Network). Розміри вимірювалися за 640×400:

- `diagram_png_2.php` — `image/png`, 2611 байт; `diagram_svg.php` — `image/svg+xml; charset=UTF-8`, 1161 байт;
- власні: `bus_png.php` — 16 901 байт, бо згладжений текст TrueType погано стискається; `bus_svg.php` — 3808 байт.

Обидва типи відповідей мають заголовок `X-Content-Type-Options: nosniff`. Розмір PNG зростає з площею полотна (1925 байт за 400×300, 5604 байти за 1200×800). Розмір SVG від розмірів майже не залежить: змінюються лише числа координат. SVG можна прочитати як XML: видно елементи `rect`, `text`, `title`, `desc`.



Рис. 4. Той самий фрагмент власної діаграми в збільшенні ×3. Ліворуч PNG: пікселі збільшено, видно сходинки й розмиття тексту. Праворуч SVG, перемальований браузером у новому масштабі: контури й літери чіткі.

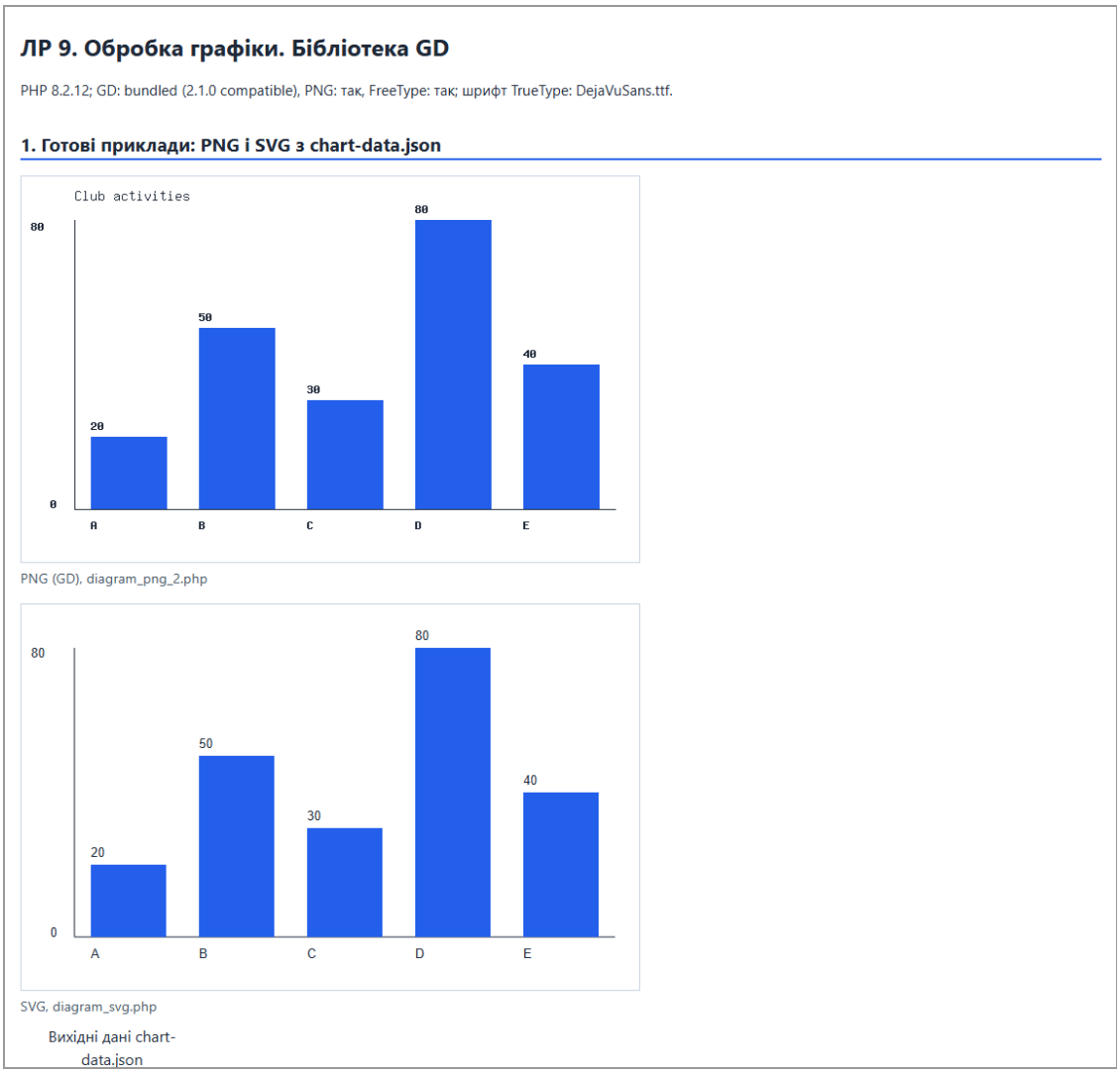


Рис. 5. `index.php` (крок 2.5): PNG і SVG поруч, таблиця вихідних даних для доступу до точних значень незалежно від кольору. Кожне `img` має змістовний `alt`. Згенерований HTML пройшов W3C без повідомлень.

Крок 3. Самостійне завдання 1: PNG-діаграма з власного файла

Схема `bus-data.json`:

- об’єкт з полями `title` (1–80 символів), `unit` (1–40) і `items` — список з 1–10 категорій;

- поля `label` і `value` збережено: мітка — 1–24 символи Unicode після `trim()` без керівних символів, тож українські літери, апостроф, пробіл, дефіс, `&` і `<` дозволені; значення — ціле число 0–1 000 000.

Файл читається лише за фіксованим серверним шляхом. Перевіряються розмір (до 64 КіБ), читання, синтаксис (`JSON_THROW_ON_ERROR`), структура й кожне поле. Будь-яка проблема дає `RuntimeException`: HTTP 500 із загальним повідомленням, а причина записується в журнал PHP (`error_log`).

```
{
    "title": "Відправлення з автовокзалу за напрямками",
    "unit": "рейсів на тиждень",
    "items": [
        {"label": "Київ", "value": 42},
        {"label": "Черкаси", "value": 35},
        {"label": "Сміла", "value": 28},
        {"label": "Умань", "value": 21},
        {"label": "Золотоноша", "value": 14},
        {"label": "Кам'янець-Подільський", "value": 4},
        {"label": "Біла Церква", "value": 9}
    ]
}
```

```
/**
 * Дані діаграми з фіксованого JSON-файла: {"title", "unit", "items": [{"label", "value"}, …]}.
 * 1–10 категорій; мітка — до 24 символів Unicode; значення — ціле 0–1000000.
 * Будь-яка проблема — RuntimeException (HTTP 500, причина — у журналі PHP).
 */
function loadBusData(): array
{
    $path = busDataFile();
    if (!is_file($path) || !is_readable($path)) {
        throw new RuntimeException("Файл даних недоступний: $path");
    }
    $size = filesize($path);
    if ($size === false || $size > BUS_MAX_FILE) {
        throw new RuntimeException('Файл даних завеликий.');
    }
    $text = file_get_contents($path);
    if ($text === false) {
        throw new RuntimeException('Не вдалося прочитати файл даних.');
    }
    try {
        $data = json_decode($text, true, 16, JSON_THROW_ON_ERROR);
    } catch (JsonException $exception) {
        throw new RuntimeException('Пошкоджений JSON: ' . $exception->getMessage(), 0, $exception);
    }
    if (!is_array($data) || array_is_list($data)) {
        throw new RuntimeException('Очікується об’єкт з полями title, unit, items.');
    }
    $title = cleanText($data['title'] ?? null, 80);
    $unit = cleanText($data['unit'] ?? null, 40);
    $items = $data['items'] ?? null;
    if ($title === null || $unit === null) {
        throw new RuntimeException('Поля title (до 80 символів) і unit (до 40) мають бути непорожніми
рядками.');
    }
    if (!is_array($items) || !array_is_list($items) || count($items) < 1 || count($items) > 10) {
        throw new RuntimeException('items має бути списком з 1–10 категорій.');
    }
    $result = [];
    foreach ($items as $i => $row) {
        $label = is_array($row) ? cleanText($row['label'] ?? null, 24) : null;
```

```
        $value = is_array($row) ? ($row['value'] ?? null) : null;
        if ($label === null || !is_int($value) || $value < 0 || $value > BUS_MAX_VALUE) {
            throw new RuntimeException('Категорія №' . ($i + 1) . ': потрібні label (1–24 символи) і ціле
value 0–' . BUS_MAX_VALUE . '.');
        }
        $result[] = ['label' => $label, 'value' => $value];
    }
    return ['title' => $title, 'unit' => $unit, 'items' => $result];
}
```

```
<?php
declare(strict_types=1);
// Самостійне завдання 1: стовпчаста PNG-діаграма з файла bus-data.json (варіант 10 — автовокзал).
require __DIR__ . '/bus-common.php';

runImage(function (): void {
    [$width, $height] = chartSize();         // GET width 400–1200, height 300–800 (готова перевірка)
    $data = loadBusData();
    $font = requireTtf();
    $measure = ttfWidth($font);
    $layout = busLayout($data['items'], $width, $height);

    $image = canvas($width, $height);
    $ink = imagecolorallocate($image, 30, 41, 59);
    $muted = imagecolorallocate($image, 90, 104, 125);
    $gridColor = imagecolorallocate($image, 226, 232, 240);
    $blue = imagecolorallocate($image, 37, 99, 235);

    [$title, $size] = fitText($data['title'], $width - 30, $measure, 15, 9);
    ttfText($image, $font, $size, 15, 28, $ink, $title);
    [$unit, $size] = fitText('Одиниця: ' . $data['unit'], $width - 30, $measure, 11, 8);
    ttfText($image, $font, $size, 15, 50, $muted, $unit);

    foreach ($layout['grid'] as $line) {
        if ($line['y'] !== $layout['bottom']) {
            imageline($image, $layout['left'] + 1, $line['y'], $layout['right'], $line['y'], $gridColor);
        }
        imageline($image, $layout['left'] - 4, $line['y'], $layout['left'], $line['y'], $ink);
        ttfText($image, $font, 10, $layout['left'] - 8, $line['y'] + 4, $ink, (string) $line['value'],
'right');
    }
    imageline($image, $layout['left'], $layout['top'] - 6, $layout['left'], $layout['bottom'], $ink);
    imageline($image, $layout['left'], $layout['bottom'], $layout['right'], $layout['bottom'], $ink);

    $labels = fitLabels($layout['bars'], $layout['slot'] - 4, $measure);
    foreach ($layout['bars'] as $i => $bar) {
        if ($bar['value'] > 0) {
            imagefilledrectangle($image, $bar['x1'], $bar['y'], $bar['x2'], $layout['bottom'] - 1, $blue);
        }
        ttfText($image, $font, 10, $bar['cx'], $bar['y'] - 6, $ink, (string) $bar['value'], 'center');
        $label = $labels[$i];
        foreach ($label['lines'] as $n => $line) {
            ttfText($image, $font, $label['size'], $bar['cx'], $layout['bottom'] + 18 + $n * 14, $ink,
$line, 'center');
        }
    }

    [$caption, $size] = fitText(authorCaption(), $width - 30, $measure, 10, 8);
    ttfText($image, $font, $size, $width - 12, $height - 12, $muted, $caption, 'right');
    sendPng($image);
});
```

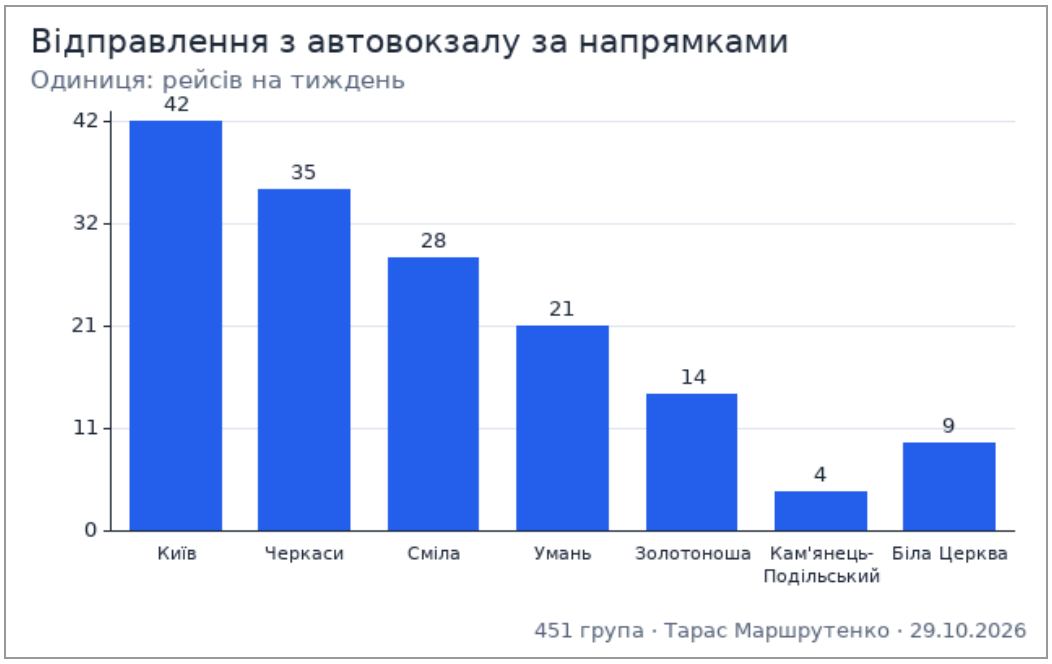


Рис. 6. `bus_png.php` (640×400). Заголовок, одиниця, сітка зі значеннями шкали, осі, нульова позначка, значення над стовпцями, українські назви напрямків (довга назва — у два рядки) і підпис «451 група · Тарас Маршрутенко · дата» (вигадане навчальне ім’я).



Рис. 7. 400×300: підписи зменшено до 8 пт, довгі скорочено з «…».

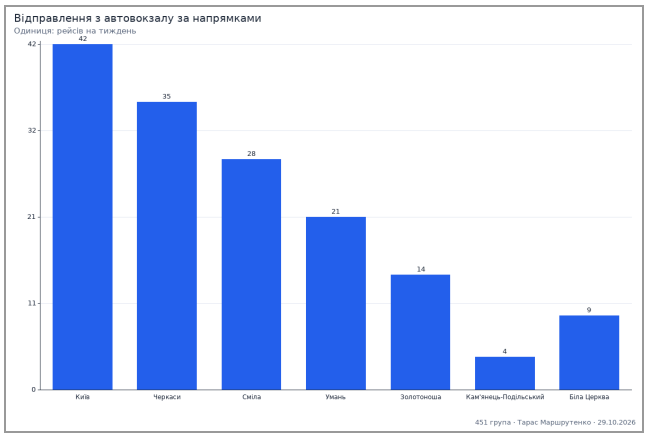


Рис. 8. 1200×800: усі підписи в один рядок.

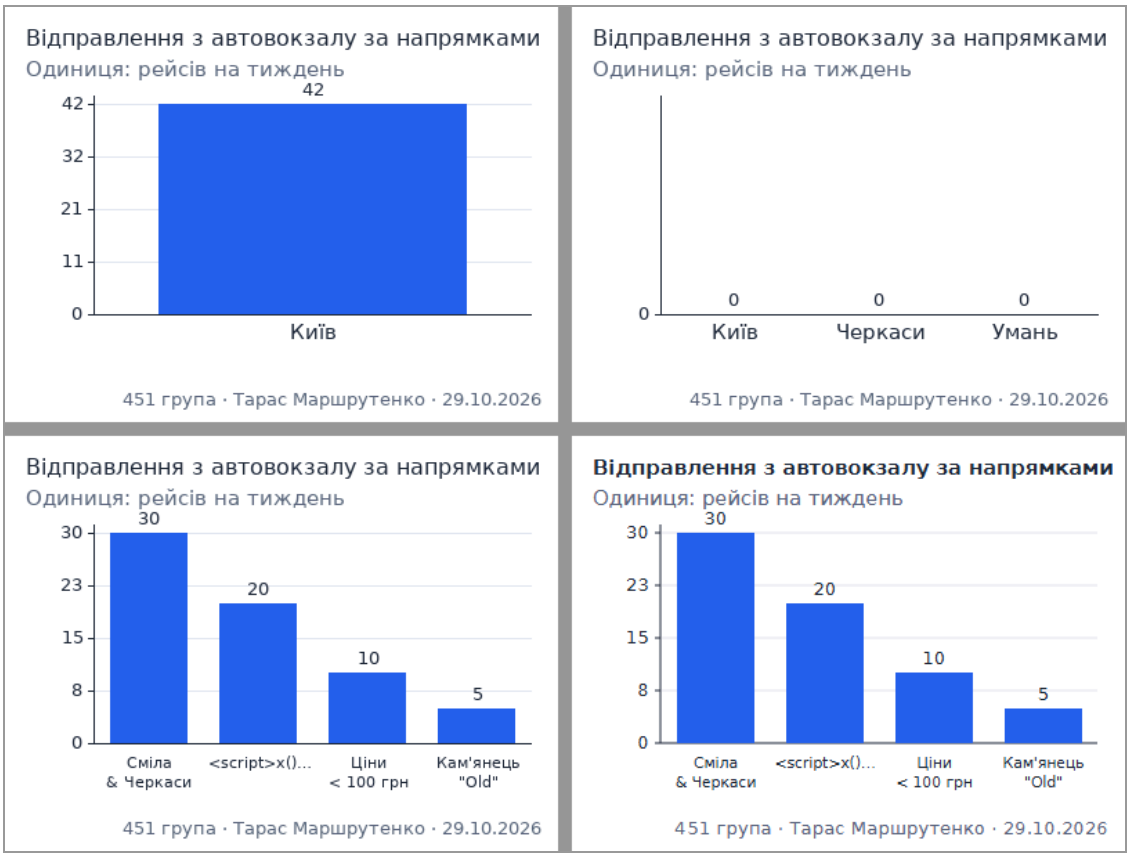


Рис. 9. Перевірки на копії даних. Угорі: один запис — один стовпець на всю висоту; усі значення 0 — нульові висоти, осі й підписи на місці, без ділення на нуль. Унизу: мітки з `&`, `<`, лапками й `<script>` у PNG (ліворуч) і SVG (праворуч) показано як текст.

Крок 4. Самостійне завдання 2: SVG-версія

`bus_svg.php` бере той самий файл і ту саму розкладку `busLayout()`. GD для SVG не потрібне: сценарій формує XML-рядок.

- Кореневий `svg` має простір імен `http://www.w3.org/2000/svg` і `viewBox`.
- `title` містить назву й одиницю, `desc` — усі значення.
- Кожен стовпець має власний `title` (підказку з точним значенням).
- Увесь текст проходить через `xmlText()` (`ENT_QUOTES | ENT_XML1`). Координати — цілі числа з розкладки, кольори — фіксовані.

Зміна JSON однаково змінює обидві версії, бо PNG і SVG будуються з одного файла. Статичний `diagram_1.svg` натомість лишається з початковими даними.

```
<?php
declare(strict_types=1);
// Самостійне завдання 2: та сама діаграма у SVG (ті самі дані, масштаб і порядок категорій; GD не
потрібне).
require __DIR__ . '/bus-common.php';

runImage(function (): void {
    [$width, $height] = chartSize();
    $data = loadBusData();
    $layout = busLayout($data['items'], $width, $height);
```

```
    $measure = static fn(string $text, int $size): float => svgWidth($text, $size);
    $text = static fn(int|float $x, int|float $y, string $value, int $size, string $extra = ''): string =>
        '<text x="' . $x . '" y="' . $y . '" font-size="' . svgFontSize($size) . '"' . $extra . '>' .
xmlText($value) . '</text>';

    $summary = implode('; ', array_map(fn(array $row): string => $row['label'] . ' — ' . $row['value'],
$data['items']));
    $svg = '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 ' . $width . ' ' . $height . '" width="' .
$width
        . '" height="' . $height . '" role="img" aria-labelledby="title desc">';
    $svg .= '<title id="title">' . xmlText($data['title'] . ' (' . $data['unit'] . ')') . '</title>';
    $svg .= '<desc id="desc">' . xmlText('Стовпчаста діаграма. ' . $summary . '.') . '</desc>';
    $svg .= '<rect width="100%" height="100%" fill="white"/>';
    $svg .= '<g font-family="DejaVu Sans, Arial, sans-serif" fill="#1e293b">';

    [$title, $size] = fitText($data['title'], $width - 30, $measure, 15, 9);
    $svg .= $text(15, 28, $title, $size, ' font-weight="bold"');
    [$unit, $size] = fitText('Одиниця: ' . $data['unit'], $width - 30, $measure, 11, 8);
    $svg .= $text(15, 50, $unit, $size, ' fill="#5a687d"');

    foreach ($layout['grid'] as $line) {
        if ($line['y'] !== $layout['bottom']) {
            $svg .= '<line x1="' . ($layout['left'] + 1) . '" y1="' . $line['y'] . '" x2="' .
$layout['right'] . '" y2="' . $line['y'] . '" stroke="#e2e8f0"/>';
        }
        $svg .= '<line x1="' . ($layout['left'] - 4) . '" y1="' . $line['y'] . '" x2="' . $layout['left'] .
'" y2="' . $line['y'] . '" stroke="#1e293b"/>';
        $svg .= $text($layout['left'] - 8, $line['y'] + 4, (string) $line['value'], 10, ' text-
anchor="end"');
    }
    $svg .= '<path d="M ' . $layout['left'] . ' ' . ($layout['top'] - 6) . ' V ' . $layout['bottom'] . ' H
' . $layout['right']
        . '" fill="none" stroke="#1e293b"/>';

    $labels = fitLabels($layout['bars'], $layout['slot'] - 4, $measure);
    foreach ($layout['bars'] as $i => $bar) {
        $svg .= '<g><title>' . xmlText($bar['label'] . ': ' . $bar['value']) . '</title>';
        $svg .= '<rect x="' . $bar['x1'] . '" y="' . $bar['y'] . '" width="' . ($bar['x2'] - $bar['x1'] +
1)
            . '" height="' . ($layout['bottom'] - $bar['y']) . '" fill="#2563eb"/></g>';
        $svg .= $text($bar['cx'], $bar['y'] - 6, (string) $bar['value'], 10, ' text-anchor="middle"');
        $label = $labels[$i];
        foreach ($label['lines'] as $n => $line) {
            $svg .= $text($bar['cx'], $layout['bottom'] + 18 + $n * 14, $line, $label['size'], ' text-
anchor="middle"');
        }
    }
    [$caption, $size] = fitText(authorCaption(), $width - 30, $measure, 10, 8);
    $svg .= $text($width - 12, $height - 12, $caption, $size, ' text-anchor="end" fill="#5a687d"');
    $svg .= '</g></svg>';

    header('Content-Type: image/svg+xml; charset=UTF-8');
    header('X-Content-Type-Options: nosniff');
    echo $svg;
});
```



Рис. 10. `bus_svg.php` у браузері Edge: той самий масштаб, порядок і підписи, що й у PNG (рис. 6). Висоти стовпців 252, 210, 168, 126, 84, 24, 54 px збігаються з PNG.

Крок 5. Самостійне завдання 3: водяний знак

Готовий `watermark.php` лишився без змін. Власний обробник `watermark_bus.php` повторює його перевірки:

- дозволений перелік ключів `sample` і `bus`, тож через HTTP передається лише ключ;
- розмір до 2 MiB;
- тип за `getimagesize()`, а не за розширенням;
- габарити від 400×200 до 2000×2000 і не більше 2 млн пікселів;
- перевірка декодування `imagecreatefrompng()`: попередження декодера `runImage()` перетворює на виняток.

Підпис «ЛР 9 · 451 група · Тарас Маршрутенко · дата» виводиться шрифтом TrueType. Кегль підбирається від 22 до 10 пт за `imagettfbbox()` так, щоб текст із полями по 16 px умістився, а висота напівпрозорої панелі визначається з тих самих меж тексту. Для PNG з прозорістю ввімкнено `imagealphablending()` і `imagesavealpha()`.

Три режими:

- **GET** — PNG у відповіді;
- **`download=1`** — `Content-Disposition: attachment; filename="watermark.png"`;
- **CLI** `php watermark_bus.php bus` — запис `watermark-bus-output.png`. Вихідне зображення не змінюється, а ключ з аргументу перевіряється за тим самим переліком.

```
/** Перевірки розміру, типу, габаритів і декодування; повертає GdImage. */
function openAllowedPng(string $path): GdImage
```

```
{
    clearstatcache(true, $path);
    if (!is_file($path) || !is_readable($path)) {
        throw new RuntimeException('Зображення недоступне.');
    }
    $bytes = filesize($path);
    if ($bytes === false || $bytes > WM_MAX_BYTES) {
        throw new RuntimeException('Файл зображення більший за 2 MiB.');
    }
    $info = getimagesize($path);
    if ($info === false || $info[2] !== IMAGETYPE_PNG) {
        throw new RuntimeException('Файл не є PNG (розширення не гарантує формату).');
    }
    [$w, $h] = $info;
    if ($w < 400 || $h < 200 || $w > 2000 || $h > 2000 || $w * $h > 2000000) {
        throw new RuntimeException("Неприпустимі габарити {$w}×{$h}: від 400×200 до 2000×2000, не більше 2
млн пікселів.");
    }
    $image = imagecreatefrompng($path);   // попередження декодера runImage() перетворює на виняток
    if ($image === false) {
        throw new RuntimeException('Не вдалося декодувати PNG.');
    }
    return $image;
}

function addWatermark(GdImage $image): void
{
    $font = requireTtf();
    $measure = ttfWidth($font);
    imagealphablending($image, true);
    imagesavealpha($image, true);
    $width = imagesx($image);
    $height = imagesy($image);
    $label = 'ЛР 9 · ' . authorCaption();
    // Найбільший кегль 22…10, за якого підпис уміщується з полями по 16 px.
    [$label, $size] = fitText($label, $width - 32, $measure, 22, 10);
    $box = imagettfbbox($size, 0, $font, $label);
    $textHeight = max($box[1], $box[3]) - min($box[5], $box[7]);
    $panelHeight = $textHeight + 24;
    $panel = imagecolorallocatealpha($image, 15, 23, 42, 35);
    $white = imagecolorallocate($image, 255, 255, 255);
    imagefilledrectangle($image, 0, $height - $panelHeight, $width - 1, $height - 1, $panel);
    ttfText($image, $font, $size, 16, $height - 12 - max($box[1], $box[3]), $white, $label);
}
```



Рис. 11. Власне навчальне PNG `images/bus.png` (800×450), намальоване GD-скриптом `make_bus_image.php`.

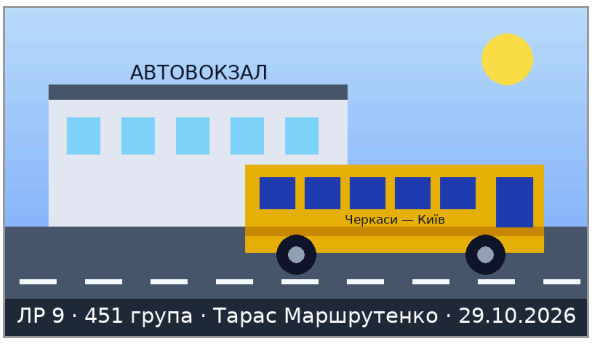


Рис. 12. `watermark_bus.php?image=bus`: український підпис на контрастній панелі.

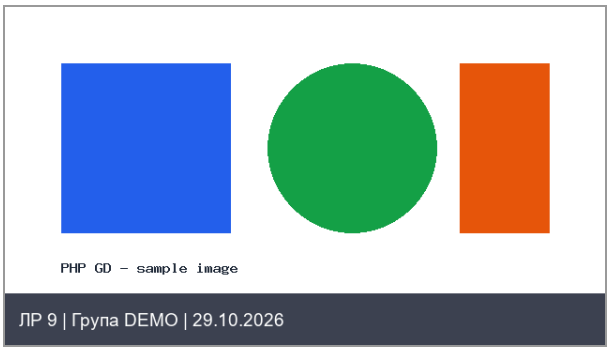


Рис. 13. Готовий `watermark.php?image=sample` (у Windows — шрифт Arial).

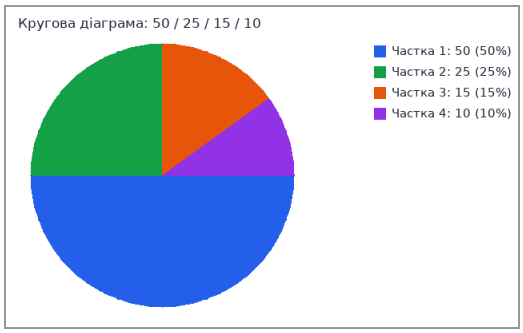


Рис. 14. `pie_png.php?values=50,25,15,10`.

5. Перевірка: сценарії й фактичні результати

№	Сценарій	Очікувано	Фактичний результат	Висновок
1	Кругова діаграма	40 %, 30 %, 30 %, без пошкодження	200 `image/png` 640×400; за пікселями 39–40 / 30 / 30 %	**Виконано**
2	Початкові дані PNG і SVG	5 стовпців, D найвищий, C ≈ 3/8 D	Висоти 75, 188, 113, 300, 150; C/D = 0,377	**Виконано**
3	400×300, 640×400, 1200×800 (та 800×500)	Правильні розміри, фігури й підписи всередині	Для `diagram_png_2`, `diagram_svg`, `bus_png`, `bus_svg` — 200 і точні розміри. На 400×300 підписи зменшено або скорочено (рис. 7)	**Виконано**
4	`width=399`, `1201`, `height=299`, `801`	400 з текстом	400 «Параметр width має бути в межах 400–1200.» / «height … 300–800.»	**Виконано**
5	`width[]=640`, `400.5`, `abc`, порожнє (також `-300`, `0640`, «640 »)	400 без попереджень	400 «Параметр width має бути цілим числом.» (для `0640` — «в межах»). Сервер працював із `display_errors=1`, попереджень немає	**Виконано**
6	Один запис у файлі	Один стовпець без ділення на нуль	Готовий і власний варіанти: 200, один стовпець (рис. 9)	**Виконано**
7	Усі значення 0	Нульові висоти, осі й підписи лишаються	200; стовпців висотою 0, підписи «0» й назви на місці (рис. 9)	**Виконано**
8	Порожній список, пошкоджений JSON, від'ємне чи рядкове value (також дробове, 11 категорій, мітка 25 символів, список замість об'єкта)	500, дані не змінюються, причина в журналі	500 «Не вдалося сформувати зображення…». У журналі: «Пошкоджений JSON: Syntax error», «items має бути списком з 1–10 категорій», «Категорія №2: потрібні label … і ціле value …». `index.php` показує повідомлення замість діаграм. Контрольні суми справжніх файлів не змінилися	**Виконано**

№	Сценарій	Очікувано	Фактичний результат	Висновок
9	Український текст і `&`, `<` у підписах SVG	Текст збережено, SVG коректний, розмітка не виконується	XML-розбір успішний; елементів `script` і атрибутів `on*` — 0; у файлі `<script>x()</script>`; W3C — без помилок	**Виконано**
10	`image=../../README.md`, `image[]=sample` (також URL, `C:/…`, інший регістр)	400 без читання файла	400 «Невідоме зображення.» для власного й готового обробника	**Виконано**
11	Перейменований не-PNG, пошкоджене, завелике	Контрольована відмова до видавання PNG	Усі випадки 500, у журналі причина: • JPEG і текст з розширенням `.png` — «Файл не є PNG»; • обрізаний PNG — «Read Error: truncated data»; • зіпсований байт — «invalid chunk type»; • 300×150, 2001×300, 1500×1500 (2,25 Мп) — «Неприпустимі габарити»; • 5,9 МБ — «більший за 2 MiB». Коректний PNG тим самим шляхом — 200	**Виконано**
12	Шрифт відсутній	Готовий — латинський підпис; український потребує налаштування	Сервер з `LAB09_FONT` на відсутній файл. Власні PNG — 500, у журналі «Не знайдено TrueType-шрифт…». SVG — 200, бо шрифт не потрібен. Готовий `watermark.php` у Windows підставляє `arial.ttf`, тож латинського запасного підпису тут не видно	**Виконано**
13	`download=1`	`image/png`, файл `watermark.png`	`Content-Disposition: attachment; filename="watermark.png"` для обох обробників; `download=2` і `download[]` — 400	**Виконано**
14	CLI-збереження	Файл існує й декодується; `sample.png` не змінено	`watermark-output.png` 640×360 і `watermark-bus-output.png` 800×450 декодуються GD; контрольні суми `sample.png` і `bus.png` не змінилися; GET нічого не записує	**Виконано**
15	HTML і SVG у валідаторі	Без помилок	W3C: `index.php` — 0 повідомлень; 5 SVG — лише інформаційне «Using the preset for SVG»	**Виконано**

Негативні перевірки виконувалися на копії `src` (`tests/work/www`), яку обслуговував окремий тестовий сервер; справжні дані не змінювалися. Невиконаних сценаріїв немає. Обмеження сценарію 12 описано в таблиці.

6. Вивід автоматичних перевірок

```
== 1. Середовище
PHP CLI: 8.2.12; php.ini: \xampp\php\php.ini
GD у CLI без параметрів: ні (рядок у php.ini: 931:;extension=gd)
```

```
GD у CLI з -d extension=gd: bundled (2.1.0 compatible); PNG так; FreeType так
phpinfo() вебсервера: Loaded Configuration File: C:\xampp\php\php.ini
   GD Support: enabled ;GD Version: bundled (2.1.0 compatible) ;FreeType Support: enabled ;PNG Support:
enabled
   діагностичну сторінку видалено: так
шрифт: C:/Windows/Fonts/DejaVuSans.ttf

== 2. Кругова діаграма
diagram_png_1.php: 200 image/png, PNG, 640 x 400, 3626 байт
   частки за кольором пікселів на колі: помаранчевий 30%, синій 39%, зелений 30%
pie_png.php?values=50,25,15,10: 200 image/png, PNG, 640 x 400, 13904 байт
pie_png.php?values=1,1: 200 image/png, PNG, 640 x 400, 9415 байт
pie_png.php?values=0,0,5: 200 image/png, PNG, 640 x 400, 11353 байт
pie_png.php?values=0,0: 400 text/plain; charset=UTF-8: values: сума має бути додатною (інакше ділення на
нуль).
pie_png.php?values=5: 400 text/plain; charset=UTF-8: values: від 2 до 6 цілих невід’ємних чисел через кому.
pie_png.php?values=1,2,3,4,5,6,7: 400 text/plain; charset=UTF-8: values: від 2 до 6 цілих невід’ємних чисел
через кому.
pie_png.php?values=1001,1: 400 text/plain; charset=UTF-8: values: кожне значення 0–1000.
pie_png.php?values=a,b: 400 text/plain; charset=UTF-8: values: від 2 до 6 цілих невід’ємних чисел через
кому.
pie_png.php?values=-1,2: 400 text/plain; charset=UTF-8: values: від 2 до 6 цілих невід’ємних чисел через
кому.
pie_png.php?values=1.5,2: 400 text/plain; charset=UTF-8: values: від 2 до 6 цілих невід’ємних чисел через
кому.
pie_png.php?values[]=1: 400 text/plain; charset=UTF-8: values: від 2 до 6 цілих невід’ємних чисел через
кому.

== 3. Розміри полотна (готові й власні діаграми)
diagram_png_2.php: 200 image/png, PNG, 640 x 400, 2611 байт
diagram_png_2.php?width=400&height=300: 200 image/png, PNG, 400 x 300, 1925 байт
diagram_png_2.php?width=640&height=400: 200 image/png, PNG, 640 x 400, 2611 байт
diagram_png_2.php?width=800&height=500: 200 image/png, PNG, 800 x 500, 3334 байт
diagram_png_2.php?width=1200&height=800: 200 image/png, PNG, 1200 x 800, 5604 байт
diagram_svg.php: 200 image/svg+xml; charset=UTF-8, SVG width="640" height="400", 1161 байт
diagram_svg.php?width=400&height=300: 200 image/svg+xml; charset=UTF-8, SVG width="400" height="300", 1160
байт
diagram_svg.php?width=640&height=400: 200 image/svg+xml; charset=UTF-8, SVG width="640" height="400", 1161
байт
diagram_svg.php?width=800&height=500: 200 image/svg+xml; charset=UTF-8, SVG width="800" height="500", 1167
байт
diagram_svg.php?width=1200&height=800: 200 image/svg+xml; charset=UTF-8, SVG width="1200" height="800",
1170 байт
bus_png.php: 200 image/png, PNG, 640 x 400, 16901 байт
bus_png.php?width=400&height=300: 200 image/png, PNG, 400 x 300, 13994 байт
bus_png.php?width=640&height=400: 200 image/png, PNG, 640 x 400, 16901 байт
bus_png.php?width=800&height=500: 200 image/png, PNG, 800 x 500, 19803 байт
bus_png.php?width=1200&height=800: 200 image/png, PNG, 1200 x 800, 21625 байт
bus_svg.php: 200 image/svg+xml; charset=UTF-8, SVG width="640" height="400", 3808 байт
bus_svg.php?width=400&height=300: 200 image/svg+xml; charset=UTF-8, SVG width="400" height="300", 3768 байт
bus_svg.php?width=640&height=400: 200 image/svg+xml; charset=UTF-8, SVG width="640" height="400", 3808 байт
bus_svg.php?width=800&height=500: 200 image/svg+xml; charset=UTF-8, SVG width="800" height="500", 3809 байт
bus_svg.php?width=1200&height=800: 200 image/svg+xml; charset=UTF-8, SVG width="1200" height="800", 3747
байт

== 4. Неприпустимі параметри розміру (HTTP 400)
bus_png.php?width=399: 400 text/plain; charset=UTF-8: Параметр width має бути в межах 400–1200. |
diagram_svg.php: 400
bus_png.php?width=1201: 400 text/plain; charset=UTF-8: Параметр width має бути в межах 400–1200. |
diagram_svg.php: 400
bus_png.php?height=299: 400 text/plain; charset=UTF-8: Параметр height має бути в межах 300–800. |
diagram_svg.php: 400
bus_png.php?height=801: 400 text/plain; charset=UTF-8: Параметр height має бути в межах 300–800. |
diagram_svg.php: 400
bus_png.php?width[]=640: 400 text/plain; charset=UTF-8: Параметр width має бути цілим числом. |
diagram_svg.php: 400
bus_png.php?width=400.5: 400 text/plain; charset=UTF-8: Параметр width має бути цілим числом. |
diagram_svg.php: 400
bus_png.php?width=abc: 400 text/plain; charset=UTF-8: Параметр width має бути цілим числом. |
diagram_svg.php: 400
```

```
bus_png.php?width=: 400 text/plain; charset=UTF-8: Параметр width має бути цілим числом. | diagram_svg.php:
400
bus_png.php?height=-300: 400 text/plain; charset=UTF-8: Параметр height має бути цілим числом. |
diagram_svg.php: 400
bus_png.php?width=0640: 400 text/plain; charset=UTF-8: Параметр width має бути в межах 400–1200. |
diagram_svg.php: 400
bus_png.php?width=640%20: 400 text/plain; charset=UTF-8: Параметр width має бути цілим числом. |
diagram_svg.php: 400

== 5. Масштаб: висоти стовпців у SVG (D = 80 найвищий, C = 30 → 3/8 висоти D)
   diagram_svg: висоти [75, 188, 113, 300, 150]; C/D = 113/300 = 0.3767 (3/8 = 0.375)
   bus_svg: висоти [252, 210, 168, 126, 84, 24, 54]; відношення до найвищого: 1.000≈1.000, 0.833≈0.833,
0.667≈0.667, 0.500≈0.500, 0.333≈0.333, 0.095≈0.095, 0.214≈0.214
   bus_png (PNG, пікселі кольору стовпця): висоти [252, 210, 168, 126, 84, 24, 54] — збігаються з SVG

== 6. Дані з файла: копія src на тестовому сервері
bus-one.json → bus_png: 200 image/png, PNG, 640 x 400, 13125 байт | bus_svg: 200 стовпців 1 | index.php:
200
bus-zeros.json → bus_png: 200 image/png, PNG, 640 x 400, 13382 байт | bus_svg: 200 стовпців 3 | index.php:
200
bus-empty.json → bus_png: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD,
дані, файли та журнал PHP. | bus_svg: 500  | index.php: 200 Дані bus-data.json недоступні або некоректні;
причину записано в журнал PHP.
bus-broken.json → bus_png: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD,
дані, файли та журнал PHP. | bus_svg: 500  | index.php: 200 Дані bus-data.json недоступні або некоректні;
причину записано в журнал PHP.
bus-negative.json → bus_png: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD,
дані, файли та журнал PHP. | bus_svg: 500  | index.php: 200 Дані bus-data.json недоступні або некоректні;
причину записано в журнал PHP.
bus-string.json → bus_png: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD,
дані, файли та журнал PHP. | bus_svg: 500  | index.php: 200 Дані bus-data.json недоступні або некоректні;
причину записано в журнал PHP.
bus-float.json → bus_png: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD,
дані, файли та журнал PHP. | bus_svg: 500  | index.php: 200 Дані bus-data.json недоступні або некоректні;
причину записано в журнал PHP.
bus-eleven.json → bus_png: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD,
дані, файли та журнал PHP. | bus_svg: 500  | index.php: 200 Дані bus-data.json недоступні або некоректні;
причину записано в журнал PHP.
bus-longlabel.json → bus_png: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте
GD, дані, файли та журнал PHP. | bus_svg: 500  | index.php: 200 Дані bus-data.json недоступні або
некоректні; причину записано в журнал PHP.
bus-list.json → bus_png: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD,
дані, файли та журнал PHP. | bus_svg: 500  | index.php: 200 Дані bus-data.json недоступні або некоректні;
причину записано в журнал PHP.
bus-markup.json → bus_png: 200 image/png, PNG, 640 x 400, 17503 байт | bus_svg: 200 стовпців 4 | index.php:
200
   XML-розбір SVG із мітками &, <, лапками й <script>: успішно; корінь svg:svg
   елементів script: 0 | атрибутів on*: 0
   підписи стовпців (title): Сміла & Черкаси: 30; <script>x()</script>: 20; Ціни < 100 грн: 10; Кам'янець
"Old": 5
   сирий фрагмент: <title>&lt;script&gt;x()&lt;/script&gt;: 20</title>
chart-one.json → diagram_png_2: 200 image/png, PNG, 640 x 400, 2185 байт | diagram_svg: 200
chart-zeros.json → diagram_png_2: 200 image/png, PNG, 640 x 400, 2215 байт | diagram_svg: 200
chart-empty.json → diagram_png_2: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення.
Перевірте GD, дані, файли та журнал PHP. | diagram_svg: 500
chart-broken.json → diagram_png_2: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення.
Перевірте GD, дані, файли та журнал PHP. | diagram_svg: 500
chart-negative.json → diagram_png_2: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення.
Перевірте GD, дані, файли та журнал PHP. | diagram_svg: 500
chart-string.json → diagram_png_2: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення.
Перевірте GD, дані, файли та журнал PHP. | diagram_svg: 500
справжні дані не змінено: bus-data.json так, chart-data.json так
журнал PHP тестового сервера (причини 500):
   items має бути списком з 1–10 категорій.
   Категорія №1: потрібні label (1–24 символи) і ціле value 0–1000000.
   Категорія №2: потрібні label (1–24 символи) і ціле value 0–1000000.
   Очікується об’єкт з полями title, unit, items.
   Очікується список із 1–10 категорій.
   Пошкоджений JSON: Syntax error
```

```
== 7. Водяний знак: вибір файла
watermark_bus.php?image=sample: 200 image/png, PNG, 640 x 360, 9267 байт
watermark_bus.php?image=bus: 200 image/png, PNG, 800 x 450, 18600 байт
watermark_bus.php?image=../../README.md: 400 text/plain; charset=UTF-8: Невідоме зображення.
watermark_bus.php?image=..%2F..%2FREADME.md: 400 text/plain; charset=UTF-8: Невідоме зображення.
watermark_bus.php?image[]=sample: 400 text/plain; charset=UTF-8: Невідоме зображення.
watermark_bus.php?image=BUS: 400 text/plain; charset=UTF-8: Невідоме зображення.
watermark_bus.php?image=http://example.com/a.png: 400 text/plain; charset=UTF-8: Невідоме зображення.
watermark_bus.php?image=C:/Windows/win.ini: 400 text/plain; charset=UTF-8: Невідоме зображення.
watermark_bus.php?image=bus&download=2: 400 text/plain; charset=UTF-8: Параметр download має бути в межах
0–1.
watermark_bus.php?image=bus&download[]=1: 400 text/plain; charset=UTF-8: Параметр download має бути цілим
числом.
watermark.php?image=../../README.md (готовий): 400 text/plain; charset=UTF-8: Невідоме зображення.
watermark.php?image[]=sample (готовий): 400 text/plain; charset=UTF-8: Невідоме зображення.
watermark.php?image=bus (готовий): 400 text/plain; charset=UTF-8: Невідоме зображення.

== 8. Водяний знак: непридатні файли (підставляються замість bus.png у копії)
jpeg-renamed.png (4372 байт): 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте
GD, дані, файли та журнал PHP.
text-renamed.png (47 байт): 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD,
дані, файли та журнал PHP.
truncated.png (904 байт): 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD,
дані, файли та журнал PHP.
corrupted.png (1507 байт): 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD,
дані, файли та журнал PHP.
small-300x150.png (523 байт): 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте
GD, дані, файли та журнал PHP.
wide-2001x300.png (2551 байт): 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте
GD, дані, файли та журнал PHP.
pixels-1500x1500.png (10002 байт): 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення.
Перевірте GD, дані, файли та журнал PHP.
heavy-noise.png (5890980 байт): 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте
GD, дані, файли та журнал PHP.
full.png (1507 байт): 200 image/png, PNG, 640 x 360, 8124 байт
журнал PHP тестового сервера (причини):
   imagecreatefrompng(): gd-png:  fatal libpng error: ID[BE]T: invalid chunk type
   imagecreatefrompng(): gd-png:  fatal libpng error: Read Error: truncated data
   Неприпустимі габарити 1500×1500: від 400×200 до 2000×2000, не більше 2 млн пікселів.
   Неприпустимі габарити 2001×300: від 400×200 до 2000×2000, не більше 2 млн пікселів.
   Неприпустимі габарити 300×150: від 400×200 до 2000×2000, не більше 2 млн пікселів.
   Файл зображення більший за 2 MiB.
   Файл не є PNG (розширення не гарантує формату).

== 9. Шрифт відсутній (сервер 8092, LAB09_FONT=відсутній файл)
bus_png.php: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD, дані, файли та
журнал PHP.
watermark_bus.php: 500 text/plain; charset=UTF-8: Не вдалося сформувати зображення. Перевірте GD, дані,
файли та журнал PHP.
bus_svg.php (GD і шрифт не потрібні): 200 image/svg+xml; charset=UTF-8, SVG width="640" height="400", 3808
байт
watermark.php (готовий): 200 image/png, PNG, 640 x 360, 7704 байт — у Windows підставляє arial.ttf
журнал: Не знайдено TrueType-шрифт або FreeType: задайте LAB09_FONT чи покладіть src/fonts/DejaVuSans.ttf.

== 10. Завантаження (download=1)
watermark.php?image=sample&download=1: Content-Type image/png; Content-Disposition: attachment;
filename="watermark.png"; PNG, 640 x 360
watermark_bus.php?image=bus&download=1: Content-Type image/png; Content-Disposition: attachment;
filename="watermark.png"; PNG, 800 x 450
без download: Content-Disposition: — (показ у браузері)

== 11. Збереження на сервері з термінала (CLI)
php watermark.php без GD: Для цього прикладу потрібне розширення GD із підтримкою PNG. Не вдалося
сформувати зображення. Перевірте GD, дані, файли та журнал PHP.
 → код 1
php -d extension=gd watermark.php: Збережено watermark-output.png
php -d extension=gd watermark_bus.php bus: Збережено watermark-bus-output.png (bus, 18600 байт)
php -d extension=gd watermark_bus.php ../secret: Невідоме зображення. Дозволено: sample, bus → код 1
src/watermark-output.png: PNG image data, 640 x 360, декодується GD: так
src/watermark-bus-output.png: PNG image data, 800 x 450, декодується GD: так
```

```
sample.png не змінено: так; bus.png не змінено: так
GET не записує файлів: так
prepare_sample.php через HTTP: 405 text/html; charset=UTF-8: Запустіть php prepare_sample.php у терміналі.;
make_bus_image.php через HTTP: 405

== 12. HTML-сторінка index.php
index.php: 200 text/html; charset=UTF-8; зображень 8, з непорожнім alt: 8
```

7. Відповіді на контрольні питання

1. **GD.** Розширення PHP для растрової графіки: створення полотен, малювання фігур і тексту, читання й кодування PNG, JPEG, GIF, WebP. Доступність перевіряють через `extension_loaded('gd')`, `gd_info()` (версія, підтримка форматів і FreeType) і секцію GD у `phpinfo()`. Треба дивитися саме на вебсервер, бо в CLI може бути інший `php.ini`. Тут GD у php.ini вимкнене, і вмикалося параметром `-d extension=gd`.
2. **GdImage.** У PHP 8 зображення — об’єкт класу `GdImage`, а не ресурс. Пам’ять звільняється, коли зникає останнє посилання, тож `imagedestroy()` не потрібна. «GD2» — назва версії бібліотеки GD 2.x і її формату файлів `.gd2`, а не окрема сучасна бібліотека. Розширення PHP так і називається `gd`, а XAMPP містить вбудовану (bundled) GD, сумісну з 2.1.
3. **Висота стовпця.** `round(value / max(1, maxValue) × plotHeight)`, верх — `bottom − barHeight`. Якщо всі значення нульові, `max(1, 0) = 1`: висоти нульові, ділення на нуль немає, осі й підписи малюються.
4. **imagecreate() і imagecreatetruecolor().** `imagecreate()` створює палітрове зображення: до 256 кольорів, перший виділений колір стає фоном. `imagecreatetruecolor()` створює повноколірне 24-бітне полотно з альфа-каналом, спочатку чорне, тож фон треба залити. Для згладженого тексту й градієнтів потрібне true color.
5. **Читання і збереження.** Спершу перевірити розмір файла й `getimagesize()` (тип, габарити, кількість пікселів). Потім декодувати `imagecreatefrompng()` або `imagecreatefromjpeg()` і перевірити, що не повернуто `false`, а попередження декодера — це помилка. `imagepng($image, $file)` зберігає у файл (перевірити результат), `imagepng($image)` з буфером виводу дає байти для відповіді. Вихідний файл не перезаписувати.
6. **PNG у відповіді.** Закодувати PNG у буфер і перевірити сигнатуру, потім надіслати `Content-Type: image/png` і байти. Будь-який HTML, BOM, пробіл чи текст попередження до або після байтів пошкоджує файл: браузер бачить зламане зображення. Помилку слід повертати окремою текстовою відповіддю з кодом 400/500.
7. **SVG і растр.** Растр — фіксована сітка пікселів: у збільшенні видно сходинки (рис. 4), розмір залежить від площі. SVG описує фігури координатами в XML, тож масштабується без пікселізації, текст лишається текстом, вміст можна читати й

індексувати. `viewBox` задає внутрішню систему координат, яку браузер відображає на будь-який фактичний розмір елемента.

8. **Екранування в SVG.** SVG — це XML. Неекранований `<` або `&` ламає документ, а розмітка в мітці могла б додати елемент `script` чи обробник подій (XSS). Числа ж перевіряють як числа: значення з введення, вставлене в атрибут, могло б вийти за межі атрибута.
9. **TrueType і imagettfbbox().** Вбудовані шрифти GD (`imagestring()`) мають лише латиницю й фіксовані розміри. TrueType-шрифт через FreeType дає UTF-8-текст, кирилицю, будь-який кегль і згладжування. `imagettfbbox()` повертає межі тексту до малювання, що дає змогу вирівняти, підібрати кегль або перенести рядок.
10. **Розмір файла і пікселі.** Після декодування кожен піксель займає близько 4 байтів. PNG розміром кілька КБ може мати 1500×1500 = 2,25 млн пікселів (фікстура: 10 КБ) і після декодування зайняти близько 9 МБ. Тому обмежують і байти файла, і габарити та їх добуток.
11. **Дозволений перелік.** Клієнт передає лише ключ (`bus`), а шлях береться із серверного масиву. Рядок `../../README.md`, URL чи абсолютний шлях не є ключем, тож сторонній файл не відкривається, і обхід каталогів неможливий.
12. **Видавання, завантаження, запис.**
    - Видавання — PNG у відповіді: браузер показує його в `img`, на сервері нічого не зберігається.
    - Завантаження — та сама відповідь із `Content-Disposition: attachment; filename=…`, і браузер зберігає файл у користувача.
    - Запис на сервері — `imagepng($image, $path)` чи `file_put_contents()`. Він змінює файлову систему сервера, тому виконується лише з CLI або через POST із CSRF-токеном, з ім’ям, згенерованим сервером.

8. Висновок

Засобами GD побудовано кругову й стовпчасту PNG-діаграми з даних і накладено водяний знак з українським підписом шрифтом TrueType. Ту саму стовпчасту діаграму сформовано як SVG. Обидві версії беруть дані з одного перевіреного JSON-файла і мають спільну розкладку, тож масштаб і висоти стовпців збігаються.

Що забезпечено:

- параметри розміру, вибір зображення й файли даних перевіряються;
- помилки повертаються текстом з кодами 400/500, а не пошкодженим зображенням;
- тип відповіді (`image/png`, `image/svg+xml`) відповідає вмісту;
- вхідні файли не перезаписуються, запис відбувається лише з термінала.

Усі 15 груп сценаріїв виконано.

Обмеження й спостереження:

- GD у XAMPP за замовчуванням вимкнене, а CLI і вебсервер можуть мати різну конфігурацію.
- Ширину тексту в SVG без шрифтових метрик можна лише оцінити, тому підписи підібрано із запасом.
- PNG важчий за SVG для тексту й більших полотен і пікселізується при збільшенні. Зате однаково відображається скрізь і не залежить від шрифтів клієнта.